INTEGRATION GUIDE

Hikvision HTTP Listening → Laravel

Direct device-to-application attendance integration, validated on a Hikvision DS-K1T321MFX

Document	Hikvision HTTP Listening → Laravel attendance integration guide
Version	1.0
Date	2026-10-31
Reference implementation	General HRMS (Laravel 12, PHP 8.3)
Hardware validated	Hikvision DS-K1T321MFX, firmware V4.61.0 build 260828
Intended reuse	Any Laravel application (e.g. DMMMSU HRMS)

Revision history

1.0	2026-10-31	First edition: architecture, device configuration, Laravel implementation, acknowledgement behaviour, network setup, testing, production checklist, troubleshooting, adaptation guide, hardware validation notes.

Contents

1. Introduction

1.1 What HTTP Listening is

1.2 Why it is useful

1.3 How it compares with other Hikvision integration methods

1.4 Why direct device-to-Laravel

2. Architecture

3. Prerequisites

3.1 Confirmed (validated on the DS-K1T321MFX)

3.2 Model- and firmware-specific (verify on your terminal)

3.3 Operational prerequisites

4. Device configuration

4.1 In the device web interface

4.2 By ISAPI (scripted configuration)

5. Laravel implementation

5.1 Route (stateless, outside the web group)

5.2 Device token: generation, storage, rotation

5.3 Controller: authenticate, parse, classify, ingest, acknowledge

5.4 Classification and history guard

5.5 Shared ingestion with idempotency

5.6 Logging the response actually sent

6. Event payload structure

7. Acknowledgement and retry behaviour

8. Network configuration

9. Testing procedure

9.1 Tests a new deployment must perform

9.2 Testing on a Windows desktop

9.3 What was actually tested on the DS-K1T321MFX

10. Production deployment checklist

11. Troubleshooting

12. Adapting the integration to another Laravel application

12.1 Reusable as-is

12.2 Must be adapted to the target application (e.g. DMMMSU HRMS)

13. Hardware validation notes

14. Appendix: polling as a recovery path and its cost

14.1 Policy in the reference implementation

14.2 Measured (benchmark, in-memory SQLite, simulated ISAPI device, pages of 30)

14.3 Estimated daily load per device count (derived from code; assumptions stated)

1. Introduction

1.1 What HTTP Listening is

Hikvision access-control and face-recognition terminals can send every event they record to an HTTP server you configure. Hikvision's ISAPI documentation calls this the **listening host** (the device web UI calls it *HTTP Listening* / *Event Alarm*; the ISAPI resource is `/ISAPI/Event/notification/httpHosts`). When an event happens, the terminal opens an HTTP connection to your server and `POST`s the event. Your server answers; a successful answer tells the terminal it may send the next event.

1.2 Why it is useful

- **Near real time.** On the validated terminal, a scan reached the database about one second later.
- **No polling load.** The server does not ask every device every minute for its recent history.
- **Store-and-forward.** The terminal keeps undelivered events and re-sends them until your server acknowledges them, so short server outages do not lose attendance.
- **No extra software.** No HikCentral, no vendor SDK, no agent at the site: the terminal talks to Laravel directly over HTTP(S).

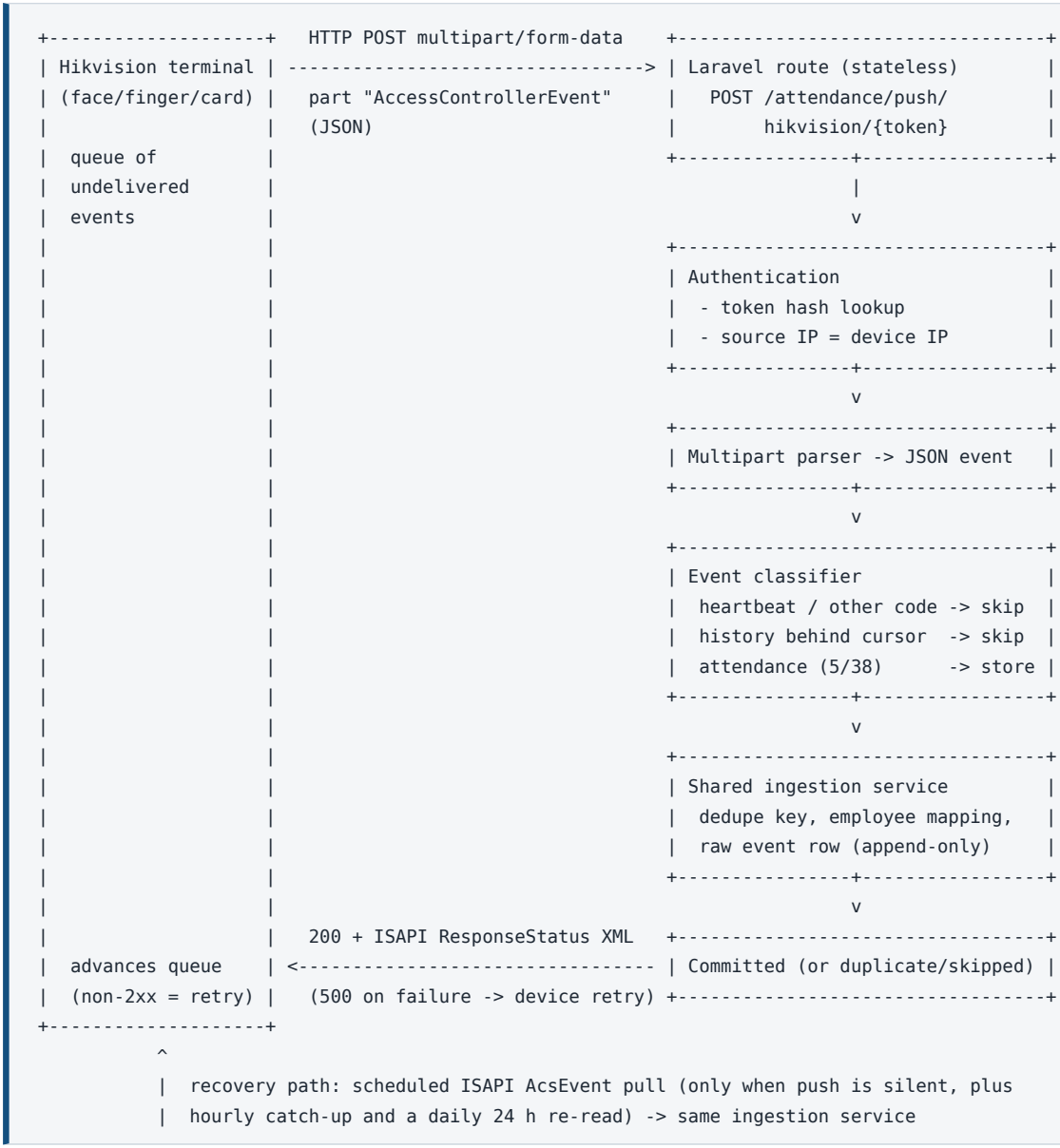
1.3 How it compares with other Hikvision integration methods

Method	Direction	What it needs	Notes
HTTP Listening (this guide)	device → server (HTTP POST)	server reachable from the device	real time; store-and-forward; documented in the public ISAPI guides
ISAPI polling (`/ISAPI/AccessControl/AcsEvent`)	server → device	device reachable from the server (same LAN or VPN)	simple and robust; latency = poll interval; re-reads history
ISAPI alertStream (`/ISAPI/Event/notification/alertStream`)	server holds a long-lived connection to the device	device reachable from the server	real time; one connection per device; no heartbeat when idle
ISUP 5.0 (EHome)	device → server, binary protocol	Hikvision ISUP SDK (native libraries, partner access) in a gateway	works across NAT without exposing the server publicly, but needs the SDK
HikCentral	device → HikCentral → your system	HikCentral licence and server	full VMS/ACS platform; another system to run

1.4 Why direct device-to-Laravel

HTTP Listening needs nothing beyond what a Laravel application already has: a route, a controller and a database. It removes a whole class of components (agents, gateways, SDKs) and their failure modes. The trade-off is network reachability: the terminal must be able to open a connection to your server (see section 8). Polling is kept as a recovery path (section 14).

2. Architecture



Key properties:

1. **One ingestion service** for every source (push, polling, manual entry, CSV). The same event arriving by push and by polling is stored once.
2. **Acknowledge after commit.** The 2xx response is sent only after the database insert has committed, or the event was recognised as a duplicate, or it was deliberately skipped.
3. **Fail loudly.** Any processing error returns a non-2xx response, so the terminal retries.

3. Prerequisites

3.1 Confirmed (validated on the DS-K1T321MFX)

Item	Confirmed detail
HTTP Listening support	`GET /ISAPI/Event/notification/httpHosts/capabilities` lists 2 host slots; protocol HTTP, HTTPS or EHome; format XML or JSON; IP or host name; port 0-65535; URL up to 128 characters; device-side authentication none, basic or MD5 digest (user name 1-32, password 8-16 characters); heartbeat 1-30 s; event mode all or list
Request format	`POST`, `multipart/form-data`, one part named `AccessControllerEvent`, `application/json`
Laravel	Laravel 12 on PHP 8.3 (any Laravel version with standard multipart parsing works the same way)
Database	MySQL/MariaDB (any database with a unique index works)

3.2 Model- and firmware-specific (verify on your terminal)

- Other models and firmware versions may differ in slot count, field names, available event codes, whether pictures are attached, and whether `attendanceStatus` is present.
- Read the device's own capability document (`.../httpHosts/capabilities`) before configuring.
- The attendance event code used here (major 5, minor 38) was observed on the validated unit; other verification methods (e.g. face match, minor 75 on the same unit) use other codes.

3.3 Operational prerequisites

- Device administrator access (web UI or ISAPI with digest authentication).
- Network path **from the terminal to the Laravel server** (section 8).
- For production: HTTPS on the server (certificate the terminal can accept), or a private network/VPN.
- Accurate device clocks (NTP recommended).

4. Device configuration

4.1 In the device web interface

The exact menu names vary by firmware; on access-control terminals the setting is usually under **Configuration → Network → Advanced Settings → HTTP Listening** (or **Event → Alarm Linkage → HTTP Listening**). Configure one host slot:

Setting	Value (example)	Notes
Event alarm IP / domain	`<server-ip-or-host>`	the address the **terminal** uses to reach Laravel; never `localhost`
URL	`/attendance/push/hikvision/<device-token>`	the token identifies and authenticates the device (section 5.2)
Port	`<port>`	e.g. 443 for HTTPS, 80/8080 for HTTP on a private network
Protocol	HTTPS (production) / HTTP (private network, testing)	
Parameter format	JSON	the receiver also accepts XML, but JSON is what was validated
Authentication	None (token in URL) or Basic/Digest	see section 10 for the trade-off
Heartbeat	30 s	heartbeats prove the push channel is alive
Event mode	List → AccessControllerEvent → minor event `0x26` (38)	sends only attendance records; omit to receive all events (door, alarm, pictures)

Save, then verify by reading the configuration back.

4.2 By ISAPI (scripted configuration)

```
PUT /ISAPI/Event/notification/httpHosts/1
Content-Type: application/xml
Authorization: Digest ...

<?xml version="1.0" encoding="UTF-8"?>
<HttpHostNotification version="2.0" xmlns="http://www.isapi.org/ver20/XMLSchema">
  <id>1</id>
  <url>/attendance/push/hikvision/DEVICE_TOKEN</url>
  <protocolType>HTTP</protocolType>
  <parameterFormatType>JSON</parameterFormatType>
  <addressingFormatType>ipaddress</addressingFormatType>
  <ipAddress>SERVER_IP</ipAddress>
  <portNo>PORT</portNo>
  <httpAuthenticationMethod>none</httpAuthenticationMethod>
  <SubscribeEvent>
    <heartbeat>30</heartbeat>
    <eventMode>list</eventMode>
```

```
    <EventList>
      <Event>
        <type>AccessControllerEvent</type>
        <minorAlarm></minorAlarm>
        <minorException></minorException>
        <minorOperation></minorOperation>
        <minorEvent>0x26</minorEvent>
        <pictureURLType>binary</pictureURLType>
      </Event>
    </EventList>
  </SubscribeEvent>
</HttpHostNotification>
```

Observed on the validated firmware:

- The `Event` element must contain the empty `minorAlarm`, `minorException` and `minorOperation` elements exactly as the device returns them on `GET`; without them the device answers `400 MessageParametersLack`.
- `GET /ISAPI/Event/notification/httpHosts/1/test` is not supported on this firmware.
- **Back up the original slot** (`GET .../httpHosts/1` and save the XML) before changing it; restore by `PUT`ting the saved XML.
- Immediately after configuration changes, quick consecutive digest requests may get a transient 401; retry after a few seconds.

5. Laravel implementation

The examples below are self-contained and use generic table names. Adapt names to your schema (section 12). They follow the pattern used by the reference implementation.

5.1 Route (stateless, outside the `web` group)

Devices cannot send CSRF tokens or cookies, so the route must not use the `web` middleware group. Register a separate route file with only the middleware you need:

```
// app/Providers/RouteServiceProvider.php (Laravel 10 structure) or bootstrap/app.php (Laravel 11+)
Route::middleware(\Illuminate\Routing\Middleware\SubstituteBindings::class)
    ->group(base_path('routes/devices.php'));
```

```
// routes/devices.php
use App\Http\Controllers\Device\HikvisionPushController;
use App\Http\Middleware\LogDevicePushRequests;

Route::post('attendance/push/hikvision/{token}', HikvisionPushController::class)
    ->middleware([LogDevicePushRequests::class, 'throttle:600,1'])
    ->where('token', '[A-Za-z0-9]{40,80}')
    ->name('attendance.device.hikvision.push');
```

`throttle:600,1` allows 10 requests per second per client IP: enough for a device draining a backlog or retrying once per second, while limiting abuse.

5.2 Device token: generation, storage, rotation

Store only a hash of the token. Show the plain token once, when it is created or rotated.

```
// in the device service
public function rotatePushToken(Device $device): string
{
    $token = \Illuminate\Support\Str::random(48);          // ~285 bits of entropy
    $device->forceFill(['push_token_hash' => hash('sha256', $token)])->save();

    return $token;                                          // show once; never log or store it
}
```

Migration column: `$table->string('push_token_hash', 64)->nullable()->unique();`

Rotation: generate a new token, update the device's HTTP Listening URL, and the old token stops working immediately (its hash is gone).

5.3 Controller: authenticate, parse, classify, ingest, acknowledge

```
namespace App\Http\Controllers\Device;

use App\Http\Controllers\Controller;
use App\Models\Device;
```

```
use App\Services\Attendance\PushNotificationService;
use Illuminate\Http\Request;
use Illuminate\Http\Response;

class HikvisionPushController extends Controller
{
    public function __construct(protected PushNotificationService $notifications) {}

    public function __invoke(Request $request, string $token): Response
    {
        $device = Device::where('push_token_hash', hash('sha256', $token))
            ->where('vendor', 'hikvision')
            ->where('is_active', true)
            ->first();

        if (! $device) {
            return $this->reject($request, 401, 'unknown token');
        }

        // Defence in depth: when the device has a fixed IP, only accept requests from it.
        if (filter_var($device->host, FILTER_VALIDATE_IP) && $request->ip() !== $device->host) {
            return $this->reject($request, 403, 'source address is not the device');
        }

        $notification = $this->readNotification($request);
        if ($notification === null) {
            return $this->reject($request, 400, 'unreadable payload');
        }

        // Throws on database errors -> Laravel renders a 500 -> the device retries.
        $outcome = $this->notifications->handle($device, $notification);   // stored|duplicate|skipped|ignored
        $request->attributes->set('push.outcome', $outcome);

        return $this->acknowledge();
    }

    /** The AccessControllerEvent part (form field or uploaded part), or a raw JSON/XML body. */
    private function readNotification(Request $request): ?array
    {
        foreach (['AccessControllerEvent', 'event_log'] as $field) {
            $value = $request->hasFile($field) ? $request->file($field)->get() : $request->input($field);
            if (is_string($value) && $value !== '') {
                return $this->decode($value);
            }
        }

        return $this->decode((string) $request->getContent());
    }

    private function decode(string $text): ?array
    {
        $text = trim($text);
        if (str_starts_with($text, '{')) {
            $data = json_decode($text, true);

            return is_array($data) ? $data : null;
        }
        if (str_starts_with($text, '<')) {
            $xml = @simplexml_load_string($text, 'SimpleXMLElement', LIBXML_NONET | LIBXML_NOCDATA);
```

```
            return $xml ? json_decode(json_encode($xml), true) : null;
        }

        return null;
    }

    private function acknowledge(): Response
    {
        $body = '<?xml version="1.0" encoding="UTF-8"?>'."\n"
            .'<ResponseStatus version="2.0" xmlns="http://www.isapi.org/ver20/XMLSchema">'
            .'<statusCode>1</statusCode><statusString>OK</statusString><subStatusCode>ok</subStatusCode>'
            .'</ResponseStatus>';

        return response($body, 200, [
            'Content-Type' => 'application/xml; charset="UTF-8"',
            'Content-Length' => (string) strlen($body),
        ]);
    }

    private function reject(Request $request, int $status, string $reason): Response
    {
        $request->attributes->set('push.outcome', 'rejected: '.$reason);

        return response('Rejected', $status);
    }
}
```

Notes:

- PHP parses `multipart/form-data` natively: a JSON part without a file name is available as `$request->input('AccessControllerEvent')`; a part sent as a file as `$request->file(...)`. Picture parts (if the subscription includes them) arrive as files and are ignored.
- `ValidatePostSize` (global middleware) and PHP's `post_max_size` bound the request size. Event-only requests are under 1 KB; pictures can be tens of KB each.

5.4 Classification and history guard

```
namespace App\Services\Attendance;

use App\Models\Device;
use Carbon\CarbonImmutable;

class PushNotificationService
{
    public function __construct(protected AttendanceIngestionService $ingestion) {}

    public function handle(Device $device, array $notification): string
    {
        if (($notification['eventType'] ?? null) !== 'AccessControllerEvent') {
            $this->markPushContact($device);                // heartBeat and other types
            return 'ignored';
        }

        $e = $notification['AccessControllerEvent'] ?? [];
        $isAttendance = (int) ($e['majorEventType'] ?? -1) === 5
```

```
            && in_array((int) ($e['subEventType'] ?? -1), $device->attendanceMinorCodes(), true)  // e.g. [38]
            && ($e['employeeNoString'] ?? '') !== '';
        if (! $isAttendance) {
            $this->markPushContact($device);                // door/alarm/other codes
            return 'ignored';
        }

        $occurredAt = CarbonImmutable::parse($notification['dateTime']);   // keeps the +08:00 offset

        // History guard: a newly configured host receives the device's stored log. Anything older
        // than the polling cursor minus an overlap is history that polling already covered.
        if ($occurredAt->lessThan($device->historyFloor())) {
            return 'skipped';
        }

        $this->markPushContact($device);

        return $this->ingestion->ingestOne($device, [
            'device_user_id'  => (string) $e['employeeNoString'],
            'device_event_id' => isset($e['serialNo']) ? (string) $e['serialNo'] : null,
            'event_time_raw'  => (string) $notification['dateTime'],
            'occurred_at'     => $occurredAt->utc(),
            'event_type'      => match ($e['attendanceStatus'] ?? null) {
                'checkIn' => 'check_in', 'checkOut' => 'check_out',
                'breakIn' => 'break_in', 'breakOut' => 'break_out',
                'overtimeIn' => 'overtime_in', 'overtimeOut' => 'overtime_out',
                default => 'unknown',
            },
            'verify_mode'     => $e['currentVerifyMode'] ?? null,
        ]);                                                  // 'stored' or 'duplicate'
    }
}
```

`historyFloor()` = the end of the last completed polling window minus the overlap (e.g. 10 minutes); if the device was never polled, use "now minus the overlap". `markPushContact()` records the time of the last push contact (write it at most once a minute); polling uses it to decide whether a device needs routine pulls (section 14).

5.5 Shared ingestion with idempotency

```
namespace App\Services\Attendance;

use App\Models\AttendanceEvent;
use App\Models\Device;
use App\Models\DeviceUserMapping;
use Illuminate\Database\QueryException;
use Illuminate\Support\Facades\DB;

class AttendanceIngestionService
{
    public function ingestOne(Device $device, array $event): string
    {
        // Stable idempotency key: the same physical event always produces the same key,
        // whichever path (push, polling, alertStream) delivers it.
        $key = hash('sha256', implode('|', [
            'device:'.$device->id, 'event', $event['device_event_id'] ?? 'none', $event['occurred_at']-
>toIso8601String(),
```

```
        ]));

        if (AttendanceEvent::where('dedupe_key', $key)->exists()) {
            return 'duplicate';
        }

        $mapping = DeviceUserMapping::where('device_id', $device->id)
            ->where('device_user_id', $event['device_user_id'])
            ->where('status', 'active')
            ->first();

        try {
            DB::transaction(fn () => AttendanceEvent::create([
                ...$event,
                'device_id'   => $device->id,
                'employee_id' => $mapping?->employee_id,               // null = unmatched, map later
                'status'      => $mapping ? 'accepted' : 'unmatched',
                'dedupe_key'  => $key,                                  // UNIQUE index
                'received_at' => now(),
            ]));
        } catch (QueryException $e) {
            // Two deliveries of the same event at the same moment: the unique index decides.
            if (AttendanceEvent::where('dedupe_key', $key)->exists()) {
                return 'duplicate';
            }
            throw $e;                                                   // genuine failure -> 500 -> retry
        }

        return 'stored';
    }
}
```

Requirements for the events table: `dedupe_key` **unique**, raw time string and timezone kept, UTC instant, device user ID, nullable employee ID (unknown users are kept as `unmatched` and mapped later), append-only (no updates to time fields).

5.6 Logging the response actually sent

Log after the response is sent, not before: a terminable middleware's `terminate()` runs after `$response->send()`. Never log the URL (it contains the token), header values or payloads.

```
namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use Symfony\Component\HttpFoundation\Response;

class LogDevicePushRequests
{
    public function handle(Request $request, Closure $next): Response
    {
        $request->attributes->set('push.started', microtime(true));
        if (app()->bound('debugbar')) {
            app('debugbar')->disable();                       // Debugbar must not touch device traffic
        }

        return $next($request);
```

```
    }

    public function terminate(Request $request, Response $response): void
    {
        $ms = (int) round((microtime(true) - $request->attributes->get('push.started')) * 1000);
        logger()->channel('daily')->info('hikvision push', [
            'ip' => $request->ip(),
            'status_sent' => $response->getStatusCode(),
            'ms' => $ms,
            'outcome' => $request->attributes->get('push.outcome'),
        ]);
    }
}
```

6. Event payload structure

Sanitized example of one part as sent by the validated terminal (`name`, `cardNo` and picture fields removed; values illustrative):

```
{
  "ipAddress": "<device-ip>",
  "portNo": 80,
  "protocol": "HTTP",
  "macAddress": "<mac>",
  "channelID": 1,
  "dateTime": "2026-10-31T17:28:04+08:00",
  "activePostCount": 1,
  "eventType": "AccessControllerEvent",
  "eventState": "active",
  "eventDescription": "Access Controller Event",
  "AccessControllerEvent": {
    "deviceName": "<device-name>",
    "majorEventType": 5,
    "subEventType": 38,
    "cardReaderNo": 1,
    "doorNo": 1,
    "employeeNoString": "<device-user-id>",
    "serialNo": 666,
    "userType": "normal",
    "currentVerifyMode": "faceOrFpOrCardOrPw",
    "attendanceStatus": "checkIn",
    "label": "Check In"
  }
}
```

Heartbeat part (every 30 s): `{"eventType": "heartBeat", "dateTime": "...", ...}` (no `AccessControllerEvent` object).

Field	Meaning
`serialNo`	the device's event counter, shared by **all** event types (door, alarm, attendance), so gaps between attendance serials are normal; together with the time it identifies an event
`dateTime`	device-local time **with UTC offset**; keep the raw string, store the UTC instant; ignore the device's time-zone name
`employeeNoString`	the person's ID **on the device** (map it to your employee)
`attendanceStatus` / `label`	the attendance key chosen on the terminal: `checkIn`, `checkOut`, `breakIn`, `breakOut` (overtime values may exist); may be missing if the terminal is not in attendance mode
`majorEventType` / `subEventType`	event code; 5/38 = the attendance records on the validated unit
`currentVerifyMode`	the **configured** verification mode (e.g. `faceOrFpOrCardOrPw`), not the method actually used for this scan

Do not assume every firmware or subscription sends identical fields. Handle missing fields (`attendanceStatus`, `serialNo`) gracefully, and log unknown shapes without storing them.

7. Acknowledgement and retry behaviour

Observed on the DS-K1T321MFX (section 13):

Server reply	Device behaviour
`200` + ISAPI `ResponseStatus` XML	event accepted; next event sent
`200` + `text/plain` `OK`	accepted
`200` + `text/html` `OK` (no explicit length)	accepted
`500`	**same event re-sent about once per second**, queue held, until a 2xx arrives
no server (connection refused)	event kept; delivered as soon as the server is back

Rules that follow:

1. **Persist before acknowledging.** A 2xx means "I have it"; after that the terminal will never send the event again. Acknowledge only after the transaction commits, the event is a known duplicate, or it is deliberately skipped (heartbeat, other codes, history).
2. **Fail with non-2xx when you cannot store.** The terminal retries, so nothing is lost.
3. **Duplicates must be safe.** Retries and parallel paths (polling) deliver the same event again; the unique idempotency key makes the second delivery a no-op that is still acknowledged.
4. **Reply fast.** The validated receiver answers in 10-110 ms. Do not call slow external services before replying; queue follow-up work instead.
5. **Outages.** While the application is down the terminal stores events and retries; after a restart it delivers them. Validated with a one-minute receiver outage.
6. **Backlog.** A newly configured host slot may receive the device's stored log first, oldest first, one request per event. Use a history guard (section 5.4) and keep acknowledging promptly.

8. Network configuration

Scenario	Works?	Requirements
Terminal and server on the same LAN	yes (validated)	server listens on its LAN address; firewall allows the terminal's IP to the port
Different sites over VPN	yes	VPN routes the terminal's subnet to the server
Public HTTPS endpoint	yes	public DNS/IP, valid certificate, firewall/WAF; restrict to the sites' public IPs where possible; never expose the terminal itself
Server only on `localhost`	no	the terminal must reach a routable address

Server binding: the web server (Nginx/Apache/IIS/`php artisan serve`) must listen on an interface the terminal can reach (`0.0.0.0` or the LAN IP), not only `127.0.0.1`. NAT: the terminal opens the connection, so only the **server** side needs an inbound rule.

Connection troubleshooting checklist:

1. From a machine on the terminal's network: `curl -X POST http(s)://<server>:<port>/attendance/push/hikvision/<token>` → expect 400 (no event) rather than a timeout.
2. Server firewall (Windows Defender Firewall / `ufw` / security groups) allows the port.
3. The device's IP settings (gateway, DNS) let it reach the server.
4. The URL path, port and protocol in the slot match the server exactly.
5. Check the server's receiver log for heartbeats every 30 s.

9. Testing procedure

9.1 Tests a new deployment must perform

1. **Reachability:** the `curl` check above from the terminal's network.
2. **Authentication:** wrong token → 401; correct token from another IP → 403 (if the IP check is on).
3. **Heartbeat:** within 30 s of saving the slot, a heartbeat request answered 200.
4. **Live scan:** scan; within seconds an event row exists with the right device user, time, status.
5. **Duplicate:** pull the same window by polling (or re-send the request) → no second row; reply 200.
6. **Outage:** stop the web server, scan, start it → the event arrives once.
7. **Backlog:** configure a fresh slot → history is skipped quickly (log shows skipped outcomes, 200).
8. **Compare with the device log:** `POST /ISAPI/AccessControl/AcsEvent?format=json` for the test period; every attendance serial is stored exactly once.
9. **No duplicates in the database:** `SELECT dedupe_key, COUNT(*) FROM attendance_events GROUP BY 1 HAVING COUNT(*) > 1` returns nothing.

9.2 Testing on a Windows desktop

1. Give the Windows PC a **fixed LAN IP** on the terminal's network (or a DHCP reservation).
2. Run the application so it listens on the LAN interface, e.g. `php artisan serve --host=0.0.0.0 --port=8022` (development only) or a local web server (Laragon/XAMPP/IIS) bound to the LAN IP. Note: `php artisan serve` drops environment-variable overrides that also appear in `.env`; configure the database in `.env` for local tests.
3. **Windows Defender Firewall:** allow inbound TCP on the chosen port for the PHP/web-server process (Private network profile), e.g. PowerShell as administrator: `New-NetFirewallRule -DisplayName "HRMS HTTP Listening" -Direction Inbound -Protocol TCP -LocalPort 8022 -RemoteAddress <terminal-ip> -Action Allow`
4. Make sure the PC does not sleep during tests (Power settings), and that the network is set to *Private*, not *Public*.
5. In the terminal's HTTP Listening slot use `http://<windows-pc-ip>:8022/attendance/push/hikvision/<token>`.
6. Run the scheduler in a second terminal (`php artisan schedule:work`) if you want the polling recovery path; it is not needed for push delivery.
7. Follow section 9.1. Watch `storage/logs/attendance-push.log` (PowerShell: `Get-Content storage\logs\attendance-push.log -Wait`).
8. When finished, restore the slot (or point it at the permanent server) so the terminal does not keep retrying against a PC that is switched off.

9.3 What was actually tested on the DS-K1T321MFX

See section 13. Heartbeats, live check-in/check-out/break events, duplicates, receiver outage and retry behaviour were tested on hardware; backlog filtering by the Laravel receiver was tested by automated tests only.

10. Production deployment checklist

- ☐ **HTTPS** on the receiver (or a private network/VPN only).
- ☐ **One token per device**, stored as a hash; rotation procedure documented.
- ☐ **Token not in access logs:** the token is part of the URL. Exclude the route from access logs or log without the path, e.g. Nginx: `location /attendance/push/hikvision/ { access_log off; try_files $uri /index.php?$query_string; }`; Apache: `SetEnvIf Request_URI "^/attendance/push/hikvision/" dontlog` with `CustomLog ... env=!dontlog`; IIS: exclude the path in logging or use Advanced Logging filters. Alternatively use the device's Basic/Digest authentication (credentials in a header, never in the URL) and validate them server-side.
- ☐ **Debugbar disabled** on device routes (and `APP_DEBUG=false` in production).
- ☐ **Source-IP check** or firewall allow-list for the terminals.
- ☐ **Rate limit** on the route (e.g. 600/min per IP).
- ☐ **Idempotency:** unique key on events; duplicate deliveries acknowledged.
- ☐ **Acknowledge after commit; non-2xx on failure.**
- ☐ **Monitoring:** alert when a device's heartbeats stop (no push contact for a few minutes).
- ☐ **Polling fallback** enabled for recovery (section 14).
- ☐ **Backlog handling** verified the first time each slot is configured.
- ☐ **Device credentials** (ISAPI user for polling) stored encrypted; dedicated non-admin account.
- ☐ **Backups** of the events table; raw events are append-only.
- ☐ **Clock sync** (NTP) on terminals and server.

11. Troubleshooting

Symptom	Likely cause / check
No requests at all	network path (section 8 checklist); server listening on `127.0.0.1` only; firewall; wrong IP/port in the slot; slot saved without the empty `minor*` elements (400 on save)
HTTP 401	token in the slot does not match a stored hash (rotated? typo?)
HTTP 403	request not from the device's registered IP (NAT, proxy: configure trusted proxies)
HTTP 404	URL path mismatch, missing route file, or the route is inside a prefixed group
HTTP 419	route is in the `web` group (CSRF); move it to the stateless group
HTTP 500	check `storage/logs/laravel.log`; the device will retry every second until fixed
Same event repeated every second	the device is not getting a 2xx: look at the status actually sent (terminable-middleware log), proxies, Debugbar, exceptions after logging
Multipart parse failure (400)	unexpected part name or format; log part names (not values)
Timestamps off by hours	store the instant from the offset in `dateTime`; do not re-interpret it in the app timezone; check NTP on the device
Events stored but `unmatched`	device user ID not mapped to an employee for that device; map it, then remap earlier events
Heartbeats arrive, events don't	event subscription excludes the code (e.g. face = 75 on some units), or `employeeNoString` empty (door events), or history guard skipping because the device clock is behind
Burst of old events after configuration	backlog drain: expected; history guard should skip them

12. Adapting the integration to another Laravel application

12.1 Reusable as-is

- Stateless receiver route with a per-device token (hash stored) and optional source-IP check.
- Multipart/JSON/XML notification reader (section 5.3).
- Classification of heartbeats, attendance codes and other events (section 5.4).
- Idempotency key (device + serial + instant) with a unique index and race handling (section 5.5).
- Acknowledge-after-commit and non-2xx-on-failure behaviour (section 7).
- Terminable logging of the status actually sent (section 5.6).
- Device configuration procedure and network checklist (sections 4 and 8).

12.2 Must be adapted to the target application (e.g. DMMMSU HRMS)

Area	What to decide
Employee/device mapping	where device user IDs live (employee field, mapping table per device?); how unknown users are handled
Attendance tables	reuse the existing raw-log table if it can hold a unique dedupe key, raw time + offset and device reference; otherwise add a new append-only table
Existing ingestion	route push events into the **existing** import/ingestion code instead of a parallel pipeline
Classification	which status values map to the target's IN/OUT/break concepts; what to do with `unknown`
Organisation scoping	campus/office (OU) rules: which devices belong to which unit; who may see/map them
Attendance calculations	DTR/daily computations should run from the stored events exactly as for other sources
Polling/reconciliation	if the target already polls devices, keep it as the recovery path and make it skip devices with a healthy push channel
Payroll	if attendance feeds payroll, make sure late or corrected events after a payroll cut-off follow the target's adjustment rules

Do not copy the reference implementation's schema or business rules blindly; port the technical pattern and plug it into the target's own models and services.

13. Hardware validation notes

Device: Hikvision DS-K1T321MFX, firmware V4.61.0 build 260828; LAN test on 2026-10-31; receiver on a macOS development machine. Results below are **observed** unless marked otherwise.

Behaviour	Status	Evidence
Request format (multipart, `AccessControllerEvent` JSON part, no pictures with a code-38 list subscription)	confirmed	standalone and Laravel receivers
Heartbeat every 30 s (`eventType: heartBeat`)	confirmed	both receivers
Live check-in / check-out / break-out delivery, ~1 s scan-to-storage	confirmed	Laravel receiver: serials 684, 687, 690, 693; matched alertStream
Break-in / break-out statuses	confirmed	standalone backlog and live (break-out live in Laravel; break-in in backlog)
2xx accepted (XML, text, html); 500 retried ~1/s until 2xx	confirmed	standalone receiver experiments
Receiver outage: event retained, delivered after restart, acknowledged events not re-sent	confirmed	standalone (serial 681) and Laravel (serial 696)
Duplicate already stored by polling acknowledged without a second row	confirmed	Laravel receiver, serial 696
Backlog drain on fresh configuration, in serial order, each once	confirmed	standalone receiver (serials 25 → 661)
Backlog filtering (history guard) **by the Laravel receiver**	**automated tests only**	the device did not replay its log during the Laravel test (it kept its delivery position when only the URL changed)
Push-health-aware polling (skips a device while its heartbeats arrive)	confirmed with real heartbeats	scheduled run reported "skipped (push channel healthy)"
HTTPS listening, Basic/Digest device-side auth	**untested**	capability reported by the device only
Other models/firmware	**untested**	

Firmware observations: `httpHosts` PUT requires the full `Event` element shape; `/httpHosts/1/test` unsupported; quick consecutive digest requests right after a configuration change may get a transient 401. An earlier receiver that logged "200" before sending saw one event repeated every second; the cause was not proven (Debugbar active on that route or an unlogged non-2xx are the likely candidates). Logging the status actually sent (section 5.6) is the safeguard.

14. Appendix: polling as a recovery path and its cost

14.1 Policy in the reference implementation

	Before	After
Routine pull	every minute, every pull-capable device	every minute **only for devices without a healthy push channel**; devices whose heartbeat/event arrived within 120 s are skipped
Catch-up for push devices	(n/a)	one pull per hour from the cursor (≈ 70-minute window)
24-hour re-read	every hour	once a day (02:30, configurable)
Page handling	all pages collected, then ingested	each page ingested as it arrives
Duplicate check	one query per event	one query per page
Heartbeat writes	(n/a)	at most one per minute per device

Configuration: `ATTENDANCE_PUSH_HEALTHY_SECONDS` (120), `ATTENDANCE_PUSH_RECONCILE_MINUTES` (60), `ATTENDANCE_LOOKBACK_DAILY_AT` (02:30), `ATTENDANCE_SYNC_OVERLAP_MINUTES` (10). Administrators can force a pull at any time (`php artisan attendance:sync-devices --force` or "Synchronize now"). Recovery triggers: push silent for more than 120 s (device offline, slot removed, receiver unreachable) → minute-by-minute catch-up from the cursor resumes automatically; the device's own store-and-forward covers receiver outages. Serial-number gaps cannot be used to detect lost events because `serialNo` counts all event types.

14.2 Measured (benchmark, in-memory SQLite, simulated ISAPI device, pages of 30)

Scenario	Device requests	DB queries	Time	Peak memory above baseline
Before: pull of 3,000 new events	100	9,008	1.60 s	9.7 MB
After: pull of 3,000 new events	100	6,108	1.40 s	1.8 MB
After: re-read of 3,000 stored events	100	107	0.19 s	2.0 MB
Before: re-read of 3,000 stored events	100	≈ 3,008 (derived from code: one duplicate check per event)	-	-

14.3 Estimated daily load per device count (derived from code; assumptions stated)

Assumptions: each device records 200 attendance events per day; a pull costs ~8 queries of bookkeeping; before the change each re-read event cost one duplicate check, after it one check per page; a routine pull window holds ~2 events (1 page); the 24-hour re-read holds 200 events (7 pages).

Devices	Before: device requests/day	After (push healthy): device requests/day	Before: DB queries/day	After: DB queries/day (pull + heartbeats)
1	1,440 + 24×7 = 1,608	24 + 7 = 31	≈ 18,000	≈ 7,500
10	16,080	310	≈ 180,000	≈ 75,000
20	32,160	620	≈ 360,000	≈ 150,000
50	80,400	1,550	≈ 900,000	≈ 375,000

Before = every-minute pull (1,440 × ~9 queries ≈ 13,000) + hourly 24 h re-read (24 × (8 bookkeeping + 200 duplicate checks) ≈ 5,000). After = heartbeats (2,880 token lookups + at most 3 writes/reads per minute for status and push contact ≈ 7,200) + hourly catch-up (≈ 260) + daily re-read (≈ 15); the scheduler's single device-list query per minute is shared by all devices. Device/network requests drop by about 98 %; database work drops by more than half (heartbeats now dominate it). Memory per pull no longer grows with the window. Pulls run sequentially in one process: an offline device can take up to ~50 s (3 attempts with timeouts), which can delay the others but never blocks them; with push primary such pulls are rare.